

**NOTES**

**Files**



## ✓ File Handling in Python

File handling is an essential skill in Python, allowing you to read, write, and manipulate files on disk. In Python, the `open()` function is used to access files. Each file can be accessed in a specific mode depending on the intended operation, like reading, writing, or appending.

### File Types

- **Text Files:** These store data in a human-readable format, typically with extensions like `.txt`. They can be opened and manipulated easily using standard string operations.
- **Binary Files:** These contain data in a format that is not human-readable, such as images (`.jpg`, `.png`) or executable files. Binary files must be handled with care as they do not support standard text operations.

### File Paths

- **Absolute Path:** This is the complete path to a file from the root of the filesystem (e.g., `C:\Documents\file.txt` on Windows or `/home/user/file.txt` on Linux).
- **Relative Path:** A path relative to the program's current working directory (e.g., `./file.txt`).

### Opening Files

To perform any file operation in Python, you must first open the file using the `open()` function. This function requires two main arguments:

1. The file path, which specifies the location of the file.
2. The access mode (determines the operations you can perform on the file).

### File Access Modes

| Mode                                                | Description                                                                                          |
|-----------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>r</code>                                      | Read only. Opens the file for reading. If the file does not exist, it raises an error.               |
| <code>r+</code>                                     | Read and write. Opens the file for both reading and writing from the beginning.                      |
| <code>w</code>                                      | Write only. Creates a new file or truncates an existing file.                                        |
| <code>w+</code>                                     | Read and write. Creates a new file or truncates an existing file for both reading and writing.       |
| <code>a</code>                                      | Append. Opens the file to append data at the end. If the file does not exist, it creates a new one.  |
| <code>a+</code>                                     | Append and read. Opens the file to append data and read it. Creates a new file if it does not exist. |
| <code>rb</code> , <code>wb</code> , <code>ab</code> | Similar to <code>r</code> , <code>w</code> , <code>a</code> modes but work with binary files.        |

### Example: Opening a File

```
# Open a file for reading
file = open("sample.txt", "r")
# Perform operations (e.g., reading)
content = file.read()
print(content)
```

```
# Close the file  
file.close()
```



```
Python is an amazing programming language.  
It is widely used in data science, web development, and automation.  
Python has a simple syntax, making it easy to learn.  
It is also known for its extensive libraries and frameworks.  
Python supports multiple programming paradigms, including procedural, object-oriented  
The community around Python is large and active, providing ample resources for learning  
With its versatility, Python is used by companies like Google, Facebook, and Netflix
```



- It opens a file named "sample.txt" in read mode.
- It reads all of its contents into memory.
- It prints those contents to the console.
- Finally, it closes the file to release resources.

## ✓ Reading from a File

Python provides several built-in methods to read content from files. Each method serves a different purpose and can be used based on the specific needs of your application. The primary methods are:

1. **read()** : The `read()` method reads the entire content of the file as a single string. This is useful when you need to load and process the entire file content at once, such as for smaller files where memory usage is not a concern. Be careful with large files! The `read()` method loads the entire content into memory, which can consume significant resources and even crash your program if the file is too large.

Example:

```
# Using read()  
file = open("sample.txt", "r")  
print(file.read()) # Reads entire file content  
file.close()
```



```
Python is an amazing programming language.  
It is widely used in data science, web development, and automation.  
Python has a simple syntax, making it easy to learn.  
It is also known for its extensive libraries and frameworks.  
Python supports multiple programming paradigms, including procedural, object-oriented  
The community around Python is large and active, providing ample resources for learning  
With its versatility, Python is used by companies like Google, Facebook, and Netflix
```



- It opens a file named "sample.txt" in read mode.
- It reads all of its contents into memory.
- It prints those contents to the console.

- Finally, it closes the file to release resources.

## ✓ Reading from a File

Python provides several built-in methods to read content from files. Each method serves a different purpose and can be used based on the specific needs of your application. The primary methods are:

1. **read()** : The `read()` method reads the entire content of the file as a single string. This is useful when you need to load and process the entire file content at once, such as for smaller files where memory usage is not a concern. Be careful with large files! The `read()` method loads the entire content into memory, which can consume significant resources and even crash your program if the file is too large.

Example:

```
# Using read()
file = open("sample.txt", "r")
print(file.read()) # Reads entire file content
file.close()
```



```
Python is an amazing programming language.
It is widely used in data science, web development, and automation.
Python has a simple syntax, making it easy to learn.
It is also known for its extensive libraries and frameworks.
Python supports multiple programming paradigms, including procedural, object-oriented
The community around Python is large and active, providing ample resources for learning
With its versatility, Python is used by companies like Google, Facebook, and Netflix
```

2. **readline()** : The `readline()` method reads one line at a time from the file. This method is used when you need to process a file line by line, which is useful for large files as it doesn't load the whole file at once. Memory-efficient for large files and is commonly used in loops to process each line individually.

Example:

```
# Using readline()
file = open("sample.txt", "r")
print(file.readline()) # Reads only the first line
file.close()
```



```
Python is an amazing programming language.
```

In this example:

- `readline()` reads the file's first line and assigns it to `line`.
- The while loop continues until `readline()` returns an empty string (end of file).
- `print(line.strip())` prints each line without extra newline characters by using `strip()`.

3. **`readlines()`** : The `readlines()` method reads all lines from a file and returns them as a list. This method is convenient for smaller files but can be problematic for larger ones because it loads the entire file content into memory at once.

Example:

```
# Using readlines()
file = open("sample.txt", "r")
lines = file.readlines() # Reads all lines into a list
print(lines)
file.close()
```

➞ ['Python is an amazing programming language.\n', 'It is widely used in data science,

◀ ▶

In this example:

- `readlines()` reads all lines and stores them as a list of strings (`lines`), with each line as an individual item
- The for loop iterates over each line, and `line.strip()` removes any extra newline characters before printing.

## ✓ Writing to a File

**`write()`** : In Python, you can use the `write()` method to add content to a file. This method writes a string to a file, creating a new file if it doesn't exist or overwriting the content if it does.

Example 1:

```
file = open("output.txt", "w") # Open a file in write mode
file.write("Hello, World!")    # Write the string "Hello, World!" to the file
file.close()                  # Close the file
```

```
# Reading output.txt to display its content
with open("output.txt", "r") as file:
    content = file.read()      # Read the content of the file
    print(content)
```

➞ Hello, World!

**`writelines()`** : The `writelines()` method in Python is used to write multiple lines to a file at once. Unlike the `write()` method, which writes a single string, `writelines()` takes an iterable (like a list or

tuple) of strings and writes them to the file without adding newline characters automatically. Therefore, you need to include newline characters (\n) in the strings if you want each entry to appear on a new line.

Example 2:

```
# Open a file in write mode
file = open("output_lines.txt", "w")

# Create a list of strings to write to the file
lines = ["Hello, World!\n", "Welcome to Python programming.\n", "This is an example of wr

# Write multiple lines to the file
file.writelines(lines)

# Close the file
file.close()

# Reading output_lines.txt to display its content
with open("output_lines.txt", "r") as file:
    content = file.read()      # Read the content of the file
    print(content)            # Display the content
```

➡ Hello, World!  
Welcome to Python programming.  
This is an example of writelines().

Explanation:

Writing to the File:

- The code opens output\_lines.txt in write mode and writes multiple lines from the lines list using writelines().
- Each string in the list ends with a newline character (\n) to ensure they appear on separate lines in the file.
- After writing, it closes the file.

Reading from the File:

- The code then opens output\_lines.txt in read mode ("r").
- It reads all content from the file using read() and stores it in the variable content.
- Finally, it prints the content to display what was written in the file.

## ✓ Appending to a file

To add content without overwriting the existing data, use append mode ("a"), which appends data to the end of the file.

```
# Open a file in append mode
file = open("output.txt", "a") # 'a' mode opens the file for appending

# Append a new line to the file
file.write("\nThis is a new line.") # Write a new line at the end of the file

# Close the file
file.close()

# Reading output.txt to display its content after appending
with open("output.txt", "r") as file:
    content = file.read() # Read the content of the file
    print(content)        # Display the content
```

➡ Hello, World!  
This is a new line.

Explanation:

Opening the File in Append Mode:

- The code opens output.txt in append mode ("a"). This means that any data written to the file will be added at the end, preserving existing content.

Appending New Content:

- The write() method is used to append the string "\nThis is a new line." to the file. The newline character (\n) ensures that this new line starts on a new line in the file.

Closing the File:

- After appending, it's important to close the file using file.close() to ensure that all changes are saved and resources are freed.

Reading and Displaying Content:

- The code then opens output.txt in read mode ("r"), reads all its content using read(), and prints it to display what is currently in the file.

## ✓ File Pointer Manipulation: seek() and tell()

The file pointer indicates where in the file you're currently reading or writing. You can control the pointer position using seek() and view its position with tell().

- **seek()** : Moves the file pointer to a specific location.

The seek(offset) method allows you to move the pointer to a specific position in the file.

```
file = open("sample.txt", "r") # Open a file in read mode
file.seek(5)                  # Move pointer to the 5th character
print(file.read())            # Read content starting from the 5th character
```

```
file.close()
```

```
➞ n is an amazing programming language.
It is widely used in data science, web development, and automation.
Python has a simple syntax, making it easy to learn.
It is also known for its extensive libraries and frameworks.
Python supports multiple programming paradigms, including procedural, object-oriented
The community around Python is large and active, providing ample resources for learning
With its versatility, Python is used by companies like Google, Facebook, and Netflix
```

The `seek(5)` method positions the pointer at the 5th character, so the output begins with "n is an amazing programming language...".

- `tell()` : Returns the current position of the file pointer.

The `tell()` method shows the current position of the file pointer, which can be useful for debugging or understanding file handling behavior.

```
file = open("sample.txt", "r")
file.seek(5)           # Move pointer to the 5th character
print(file.tell())     # Show current position (should be 5)
file.close()
```

```
➞ 5
```

## ✓ Working with Binary Files

Binary files are used to store data that isn't plain text, such as images, audio, or executable files. To handle binary data, open the file in binary mode using `rb` for reading and `wb` for writing.

```
# Writing bytes to a binary file
with open("binaryfile.bin", "wb") as file: # Open a binary file in write mode
    file.write(b'\x00\xff\x1A')           # Write binary data to the file
```

- The `b` prefix before `\x00\xff\x1A` indicates that these are byte literals.
- This code will create `binaryfile.bin` and store the specified byte sequence.

Reading binary files in text mode or vice versa may lead to errors or data corruption.

## ✓ Closing Files with `close()`

It's essential to close files after operations to release system resources. Python provides a `with` statement, which automatically handles file closure when the block ends, reducing the chance of



errors.

Example: Using with for automatic file closure

```
with open("sample.txt", "r") as file: # Open a file in read mode
    content = file.read()             # Read file content
    print(content)                     # Print content to the console
# File is automatically closed after this block
```



Python is an amazing programming language.  
It is widely used in data science, web development, and automation.  
Python has a simple syntax, making it easy to learn.  
It is also known for its extensive libraries and frameworks.  
Python supports multiple programming paradigms, including procedural, object-oriented.  
The community around Python is large and active, providing ample resources for learning.  
With its versatility, Python is used by companies like Google, Facebook, and Netflix.

- The with statement opens sample.txt and reads its content. Once the block is exited, the file automatically closes.
- This approach is generally safer than manually calling file.close() and is especially useful when working with large or multiple files.

## ✓ Working with CSV Files

CSV (Comma-Separated Values) files are commonly used for storing tabular data. Python's built-in csv module provides functionality for reading and writing CSV files.

## ✓ Reading a CSV File

To read a CSV file, use the csv.reader method, which reads the file line-by-line.

```
import csv

# Create a sample CSV file
with open('data.csv', 'w', newline='') as file:
    writer = csv.writer(file)
    writer.writerow(['name', 'age', 'city'])
    writer.writerow(['John', '30', 'New York'])
    writer.writerow(['Alice', '25', 'Los Angeles'])

# Reading the CSV file
with open('data.csv', 'r') as file:
    csv_reader = csv.reader(file)
    header = next(csv_reader) # Skip the header row
    for row in csv_reader:
        print(f"Name: {row[0]}, Age: {row[1]}, City: {row[2]}")
```

```
➡ Name: John, Age: 30, City: New York
   Name: Alice, Age: 25, City: Los Angeles
```

Explanation:

- The `writer.writerow()` method writes individual rows to `data.csv`.
- The `next(csv_reader)` skips the header row.
- This code reads each row and displays the name, age, and city fields.

## ✓ Using DictReader for Named Columns

DictReader reads CSV files where each row is stored as a dictionary with the column names as keys.

```
# Using DictReader for named columns
with open('data.csv', 'r') as file:
    csv_dict_reader = csv.DictReader(file)
    for row in csv_dict_reader:
        print(f"Name: {row['name']], Age: {row['age']], City: {row['city']]")
```

```
➡ Name: John, Age: 30, City: New York
   Name: Alice, Age: 25, City: Los Angeles
```

Explanation:

- DictReader automatically interprets the header row as dictionary keys.
- Each row is represented as a dictionary, making it easier to access data by column name.

## ✓ Writing to a CSV File

To write data to a CSV file, you can use `csv.writer`.

```
import csv

# Data to be written
data = [
    ['Name', 'Age', 'City'],
    ['John', '30', 'New York'],
    ['Alice', '25', 'Los Angeles']
]

# Writing to CSV
with open('output.csv', 'w', newline='') as file:
    csv_writer = csv.writer(file)
    csv_writer.writerows(data) # Write all rows of data to the CSV file

# Reading and displaying the content of the CSV file
with open('output.csv', 'r') as file:
```

```
csv_reader = csv.reader(file)
for row in csv_reader:
    print(row) # Print each row from the CSV file
```

```
➡ ['Name', 'Age', 'City']
   ['John', '30', 'New York']
   ['Alice', '25', 'Los Angeles']
```

Explanation:

Importing the CSV Module:

- The csv module is imported to handle CSV file operations.

Data Preparation:

- A list of lists named data is created, where each inner list represents a row in the CSV file.

Writing to CSV:

- The with open('output.csv', 'w', newline='') as file: statement opens (or creates) a file named output.csv in write mode ('w').
- The csv.writer(file) creates a writer object that will be used to write data to the CSV.
- The csv\_writer.writerows(data) method writes all rows from the data list into the CSV file.

Reading and Displaying Content:

- After writing, another with open('output.csv', 'r') as file: statement opens the same CSV file in read mode ('r').
- A csv.reader(file) object is created to read the contents of the CSV.
- A loop iterates through each row returned by csv\_reader, and each row is printed.

## ✓ Using DictWriter

DictWriter allows writing dictionaries to CSV files, which can be helpful if your data is already organized as dictionaries.

```
import csv

# Dictionary data to be written
data_dict = [
    {'name': 'John', 'age': 30, 'city': 'New York'},
    {'name': 'Alice', 'age': 25, 'city': 'Los Angeles'}
]

# Writing to CSV with DictWriter
with open('output.csv', 'w', newline='') as file:
    fieldnames = ['name', 'age', 'city']
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader() # Write header row
    writer.writerows(data_dict) # Write the dictionary data
```

```
# Reading and displaying the content of the CSV file
with open('output.csv', 'r') as file:
    csv_reader = csv.reader(file)
    for row in csv_reader:
        print(row) # Print each row from the CSV file
```

```
➡ ['name', 'age', 'city']
   ['John', '30', 'New York']
   ['Alice', '25', 'Los Angeles']
```

Explanation:

Importing the CSV Module:

- The csv module is imported to facilitate CSV file operations.

Data Preparation:

- A list of dictionaries named data\_dict is created, where each dictionary represents a row in the CSV file with keys corresponding to column names.

Writing to CSV:

- The with open('output.csv', 'w', newline="") as file: statement opens (or creates) a file named output.csv in write mode ('w').
- The fieldnames list specifies the order of columns in the CSV.
- A csv.DictWriter object is created with the specified fieldnames.
- The writer.writeheader() method writes the header row (column names) to the CSV file.
- The writer.writerows(data\_dict) method writes all rows from the list of dictionaries into the CSV file.

## ✓ Working with JSON Files

JSON (JavaScript Object Notation) is a lightweight data format that's easy to read and write. Python's `json` module provides methods for encoding and decoding JSON data.

## ✓ Reading a JSON File

To read a JSON file, use `json.load()`, which loads JSON data as a Python dictionary.

```
import json

# Create a sample JSON file
data = {
    'name': 'John',
    'age': 30,
    'city': 'New York'
}
```

```
with open('data.json', 'w') as file:
    json.dump(data, file, indent=4)
```

```
# Reading the JSON file
with open('data.json', 'r') as file:
    data = json.load(file)
    print(f"Name: {data['name']}")
    print(f"Age: {data['age']}")
    print(f"City: {data['city']}")
```

```
➡ Name: John
   Age: 30
   City: New York
```

Explanation:

- `json.dump(data, file)` writes the dictionary data as JSON in `data.json`.
- `json.load(file)` reads the JSON file and converts it to a dictionary.
- Each key-value pair in the dictionary can be accessed by its key, like `data['name']`.

## ✓ Writing to a JSON File

To write a Python dictionary to a JSON file, use `json.dump()`.

```
# Data to be written
data = {
    'name': 'Alice',
    'age': 25,
    'city': 'Los Angeles'
}
```

```
# Writing to a JSON file
with open('output.json', 'w') as file:
    json.dump(data, file, indent=4) # `indent=4` makes the JSON pretty-printed
```

```
# Reading and displaying the content of the JSON file
with open('output.json', 'r') as file:
    loaded_data = json.load(file) # Load the JSON data from the file
    print(f"Name: {loaded_data['name']}")
    print(f"Age: {loaded_data['age']}")
    print(f"City: {loaded_data['city']}")
```

```
➡ Name: Alice
   Age: 25
   City: Los Angeles
```

Explanation:

- `json.dump()` writes the dictionary data to `output.json`.
- The `indent=4` parameter formats the JSON with indentation, making it easier to read.

## ✓ XML Files

XML (eXtensible Markup Language) is used to store and transfer data in a structured format. The `xml.etree.ElementTree` module in Python provides tools for parsing and creating XML documents.

## ✓ Reading an XML File

To read an XML file, we use `ET.parse()` to load it and `.getroot()` to access the root element.

```
import xml.etree.ElementTree as ET

# Create an XML file for testing
xml_content = '''<data>
    <person name="Vishwa" age="30">New York</person>
    <person name="Arpit" age="25">Los Angeles</person>
</data>'''

with open('data.xml', 'w') as file:
    file.write(xml_content)

# Reading the XML file
tree = ET.parse('data.xml')
root = tree.getroot()

# Iterating through elements
for child in root:
    print(f"Tag: {child.tag}, Text: {child.text}")
    for key, value in child.attrib.items():
        print(f"Attribute {key}: {value}")
```

```
➡ Tag: person, Text: New York
   Attribute name: Vishwa
   Attribute age: 30
   Tag: person, Text: Los Angeles
   Attribute name: Arpit
   Attribute age: 25
```

Reading the XML File:

- The `ET.parse('data.xml')` function reads the XML file and creates an `ElementTree` object.
- The `getroot()` method retrieves the root element of the XML tree.

Iterating Through Elements:

- A loop iterates through each child element of the root.
- For each child, it prints the tag name and text content.
- Another loop iterates through each attribute of the child element, printing its key and value.

## ✓ Context Managers

Context managers in Python provide a clean way to handle resource management, particularly with files. The `with` statement ensures that files are properly closed after usage.

```
# Writing to a file using a context manager
with open('file.txt', 'w') as file:
    file.write("Hello, World!")
```

```
# Reading the file using a context manager
with open('file.txt', 'r') as file:
    content = file.read()
    print(content)
```

➞ Hello, World!

Explanation:

- `with open(...)` automatically closes the file after reading or writing, avoiding potential memory leaks.

## ✓ Logging

The logging module in Python allows you to create logs at different levels (DEBUG, INFO, WARNING, ERROR, CRITICAL). Logs help with tracking application events, especially for debugging.

## ✓ Basic Logging Setup

Basic logging configuration can send messages to a file.

```
import logging

# Configure logging
logging.basicConfig(
    level=logging.DEBUG, # Set the logging level
    format='%(asctime)s - %(levelname)s - %(message)s', # Define the log message format
    filename='app.log', # Specify the log file name
    filemode='w' # Set mode to 'w' to overwrite the log file each time
)

# Different logging levels
logging.debug('This is a debug message')
logging.info('This is an info message')
logging.warning('This is a warning message')
logging.error('This is an error message')
```

```
logging.critical('This is a critical message')
```

```
# Display log file content (optional)
with open('app.log', 'r') as log_file:
    print(log_file.read())
```

```
➡ WARNING:root:This is a warning message
   ERROR:root:This is an error message
   CRITICAL:root:This is a critical message
2024-11-19 07:19:48,597 - my_app - DEBUG - Debug message from custom logger
2024-11-19 07:19:48,603 - my_app - INFO - Info message from custom logger
```

## ✓ Custom Logger

Custom loggers allow more flexibility, like logging to both console and file with different handlers.

```
# Creating a custom logger
logger = logging.getLogger('my_app')
logger.setLevel(logging.DEBUG)

# Create handlers for file and console
file_handler = logging.FileHandler('app.log')
console_handler = logging.StreamHandler()

# Create and set formatter
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
file_handler.setFormatter(formatter)
console_handler.setFormatter(formatter)

# Add handlers to logger
logger.addHandler(file_handler)
logger.addHandler(console_handler)

# Logging messages
logger.debug('Debug message from custom logger')
logger.info('Info message from custom logger')

➡ 2024-11-19 07:19:48,597 - my_app - DEBUG - Debug message from custom logger
   DEBUG:my_app:Debug message from custom logger
   2024-11-19 07:19:48,603 - my_app - INFO - Info message from custom logger
   INFO:my_app:Info message from custom logger
```

## ✓ Debugging Techniques

Python offers various debugging strategies, including `pdb` for step-by-step debugging, `try-except` blocks for error handling, `print` statements, and logging.



## ✓ Using pdb for Step-by-Step Debugging

pdb is the Python debugger module that lets you set breakpoints and inspect code line-by-line. It is a powerful tool for diagnosing issues in your code interactively.

```
import pdb

def complex_function(x, y):
    result = x + y
    pdb.set_trace() # Pauses execution here
    result = result * 2
    return result

# Call the function
print(complex_function(3, 5))

# Common pdb commands:
# n (next line)
# s (step into)
# c (continue)
# p variable_name (print variable)
# l (list source code)
# q (quit debugger)
```



PYDEV DEBUGGER WARNING:  
sys.settrace() should not be used when the debugger is being used.  
This may cause the debugger to stop working correctly.  
If this is needed, please check:  
<http://pydev.blogspot.com/2007/06/why-cant-pydev-debugger-work-with.html>  
to see how to restore the debug tracing back correctly.  
Call Location:  
File "/usr/lib/python3.10/bdb.py", line 336, in set\_trace  
sys.settrace(self.trace\_dispatch)

```
> <ipython-input-22-a2aa2250540b>(6)complex_function()
4     result = x + y
5     pdb.set_trace() # Pauses execution here
----> 6     result = result * 2
7     return result
8
```

```
ipdb> n
> <ipython-input-22-a2aa2250540b>(7)complex_function()
5     pdb.set_trace() # Pauses execution here
6     result = result * 2
----> 7     return result
8
9 # Call the function
```

```
ipdb> c
```

PYDEV DEBUGGER WARNING:  
sys.settrace() should not be used when the debugger is being used.  
This may cause the debugger to stop working correctly.

If this is needed, please check:

<http://pydev.blogspot.com/2007/06/why-cant-pydev-debugger-work-with.html>

to see how to restore the debug tracing back correctly.

Call Location:

```
File "/usr/lib/python3.10/bdb.py", line 347, in set_continue
    sys.settrace(None)
```

16

## ✓ Using try-except for Debugging and Error Handling

The try-except block catches exceptions, making it easier to debug and handle errors gracefully.

```
def debug_function():
    try:
        result = 1 / 0 # Intentional error
    except Exception as e:
        print(f"Error occurred: {e}")
        import traceback
        print(f"Traceback: {traceback.format_exc()}")
```

# Call the function

```
debug_function()
```

```
⇒ Error occurred: division by zero
Traceback: Traceback (most recent call last):
  File "<ipython-input-23-3605ed5c6b91>", line 3, in debug_function
    result = 1 / 0 # Intentional error
ZeroDivisionError: division by zero
```

Explanation:

- The try block contains code that may raise an exception.
- The except block catches exceptions, prints error details, and provides a traceback for debugging.

## ✓ Debugging with Print Statements

Print statements are a simple debugging tool to display variable values and program states.

```
def debug_with_print():
    print("[DEBUG] Starting function")
    result = 3 + 5
    print(f"[DEBUG] Result: {result}")
    return result
```

# Call the function

```
debug_with_print()
```

```
➞ [DEBUG] Starting function  
[DEBUG] Result: 8  
8
```

Explanation:

- [DEBUG] labels make it easier to spot debug output in console logs.
- This approach is simple but effective for basic debugging.

## ✓ Using Logging for Debugging

Logging provides more flexibility and control than print statements.

```
# Setup basic logging  
logging.basicConfig(level=logging.DEBUG)  
logger = logging.getLogger(__name__)  
  
def debug_with_logging():  
    logger.debug("Starting function")  
    result = 3 + 5  
    logger.debug(f"Result: {result}")  
    return result  
  
# Call the function  
debug_with_logging()
```

```
➞ 8
```

Explanation:

- Using logging for debugging allows level-based filtering and directs output to various destinations, not just the console.
- The DEBUG level is especially useful for debugging without cluttering the output.

# THANK YOU



**Stay updated with us!**



[Vishwa Mohan](#)



[Vishwa Mohan](#)



[vishwa.mohan.singh](#)



[Vishwa Mohan](#)